



TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ - VIỄN THÔNG

LẬP TRÌNH ĐA NỀN TẢNG

SQLite và Drift - Quản lý cơ sở dữ liệu quan hệ

SINH VIÊN THỰC HIỆN:

NGUYỄN VIỆT HOÀNG 22KTMT1 106220216

LÊ MINH NHẬT 22DT2 106220066

GVHD: TS. NGUYỄN DUY NHẬT VIỄN

ĐÀ NẴNG, 10/2025

BẢNG PHÂN CÔNG VIỆC TRONG NHÓM

STT	HỌ VÀ TÊN	NHIỆM VỤ	KHỐI LƯỢNG
17	NGUYỄN VIỆT HOÀNG	Tìm hiểu về cách tạo ứng dụng quản lý danh sách sản phẩm với SQLite. Mô tả Migrate sang Drift và so sánh ưu điểm, nhược điểm. Demo các truy vấn phức tạp, mối quan của SQLite thô và Drift. Hoàn thiện báo cáo và slide tất các nội dung.	80%
07	LÊ MINH NHẬT	Tìm hiểu nội dung về kiểm tra hiệu năng và tối ưu hóa CSDL.	20%

Link code github: <https://github.com/VietHoang301/ltdntchude2>

NỘI DUNG

1. SQLite là gì?
2. Ứng dụng quản lý sản phẩm với SQLite
3. Drift và so sánh ưu nhược điểm khi Migrate to Drift
4. Demo các truy vấn phức tạp, mối quan hệ của SQLite thô (sqlite) và Drift
5. Kiểm tra hiệu năng và tối ưu hóa

1. SQLite là gì?

SQLite là một hệ quản trị cơ sở dữ liệu quan hệ, không cần mô hình client - server nên rất nhỏ gọn, nghĩa là toàn bộ cơ sở dữ liệu được lưu trữ trong một tệp tin duy nhất.



Ưu điểm và nhược điểm

Ưu điểm:

- Không cần mô hình **client – server** nên thao tác nhanh.
- SQLite không cần phải cấu hình tức là bạn không cần phải cài đặt.
- Database được lưu trữ trên một tập tin duy nhất.
- SQLite nhỏ gọn, rất đơn giản, dễ dàng sử dụng và truy xuất dữ liệu.

Nhược điểm:

- Trong cùng một thời điểm SQLite chỉ có 1 người có thể ghi dữ liệu.
- Không đáp ứng tốt khi xử lý trên một khối lượng dữ liệu lớn, phát sinh liên tục.

2. Ứng dụng quản lý sản phẩm với SQLite

Ứng dụng dùng **Flutter** (ngôn ngữ lập trình dart) với **SQLite**:

+ **Các chức năng chính:** Thêm sản phẩm, Sửa sản phẩm, Xóa sản phẩm.

Code



```

  model
  product.dart

  screen
  add_product_screen.dart
  detail_product_screen.dart
  edit_product_screen.dart
  product_screen.dart

  service
  product_service.dart

main.dart
```

Demo App Quản lý sản phẩm

QUẢN LÝ DANH SÁCH SẢN PHẨM

TV

Giá:100.000 VND

TU LANH

Giá:1000.000 VND

←

Thêm sản phẩm

Mã sản phẩm

Tên sản phẩm

Giá

Số lượng

Lưu

QUẢN LÝ DANH SÁCH SẢN PHẨM

TV

Giá:100.000 VND

TU LANH

Giá:1000.000 VND

iPhone 17 Promax

Giá:17000.000 VND

3. Drift và so sánh ưu nhược điểm khi Migrate to Drift

Drift (trước đây gọi là Moor) là một thư viện ORM (Object-Relational Mapping) mạnh mẽ xây dựng trên SQLite.

Lý do nên chuyển sang Drift

- Truy cập an toàn về kiểu
- Trình tạo truy vấn hoàn chỉnh
- Stream tự động cập nhật
- Hỗ trợ di chuyển CSDL
- Hiệu suất cao và đa nền tảng
- Hỗ trợ đa phương ngữ



Các bước thực hiện khi chuyển sang Drift

B1: Thêm Dependencies vào file pubspec.yaml:

```
1 dependencies:
2   drift: ^2.29.0
3
4 dev_dependencies:
5   drift_dev: ^2.29.0
6   build_runner: ^2.9.0
```

B2: Tạo Lớp Database:

Định nghĩa một class AppDatabase, schemaVersion, MigrationStrategy

```
1 import 'package:drift/drift.dart';
2 part 'database.g.dart';
3
4 @DriftDatabase(include: {'schema.drift'})
5 class AppDatabase extends _$AppDatabase {
6   AppDatabase() : super(_openDatabase());
7
8   @override
9   int get schemaVersion => throw UnimplementedError(
10     'todo: The schema version used by your existing database',
11   );
12
13   @override
14   MigrationStrategy get migration {
15     return MigrationStrategy(
16       onCreate: (m) async {
17         await m.createAll();
18       },
19       onUpgrade: (m, from, to) async {
20         // This is similar to the `onUpgrade` callback from sqflite. When
21         // migrating to drift, it should contain your existing migration logic.
22         // You can access the raw database by using `customStatement`
23       },
24       beforeOpen: (details) async {
25         // This is a good place to enable pragmas you expect, e.g.
26         await customStatement('pragma foreign_keys = ON;');
27       },
28     );
29   }
30
31   static QueryExecutor _openDatabase() {
32     throw UnimplementedError(
33       'todo: Open database compatible with the one that already exists',
34     );
35   }
36 }
```

Các bước thực hiện khi chuyển sang Drift

B3: Chạy Build Runner:

trong terminal để Drift tạo file **database.g.dart** tương ứng.

B4: Tái cấu trúc (Refactor) các hàm CRUD:

Create -> Read -> Read (phản ứng) -> Update.

Loại bỏ hoàn toàn mã lặp lại (**toMap/fromMap**).

Cung cấp các tính năng mạnh mẽ như stream phản ứng.

```
1 //Thêm sản phẩm
2 Future<int> createProduct(ProductsCompanion product) {
3     return into(products).insert(product);
4 }
5 //Lấy danh sách sản phẩm 1 lần
6 Future<List<Product>> getProducts() {
7     return select(products).get();
8 }
9
10 //Lấy danh sách sản phẩm phản ứng
11 Stream<List<Product>> watchProducts() {
12     return select(products).watch();
13 }
14
15 //Cập nhật sản phẩm
16 Future<bool> updateProduct(Product product) {
17     return update(products).replace(product);
18 }
```

So sánh ưu điểm và nhược điểm

Sqflite

- **An toàn về kiểu:** Đây là ưu điểm lớn nhất. Khi truy vấn, sẽ nhận về các lớp kết quả đã được định nghĩa rõ ràng. Nếu bạn gõ sai tên cột hoặc kiểu dữ liệu, trình biên dịch sẽ báo lỗi ngay lập tức.
- **Stream tự động cập nhật:** qua phương thức `.watch()`. Bất cứ khi nào dữ liệu thay đổi, stream sẽ tự động phát ra dữ liệu mới, giúp việc cập nhật UI trở nên dễ dàng.
- **SQL tự động và Giảm mã lặp:** tự động sinh ra các hàm chuyển đổi `fromMap/toMap`.
- **Hỗ trợ Di chuyển:** được tích hợp sẵn.

Ưu điểm:

Drift

- **Không an toàn về kiểu:** Tự phân tích dữ liệu Map thô. Mọi lỗi chỉ được phát hiện khi ứng dụng chạy đến đoạn mã đó.
- **Stream tự triển khai thủ công:** Nếu muốn UI tự cập nhật, phải tự quản lý state và gọi thông báo cập nhật sau mỗi thao tác CSDL, tốn nhiều công sức và mã lặp lại.
- **SQL thô bằng chuỗi (String):** rất dễ sai sót. Tự viết mã lặp lại để chuyển đổi **Map** sang đối tượng (**Object**) và ngược lại cho mọi bảng.
- **Không hỗ trợ di chuyển:** Toàn bộ logic di chuyển và việc kiểm thử đều thực hiện thủ công.

So sánh ưu điểm và nhược điểm

Sqflite

Nhược điểm:

Drift

- **Phức tạp khi thiết lập:** phải cài đặt **drift_dev** và **build_runner**, cấu hình lớp **DriftDatabase**.
- **Phải build_runner:** Khi thay đổi cấu trúc bảng hoặc thêm một truy vấn SQL có tên trong file **.drift**.

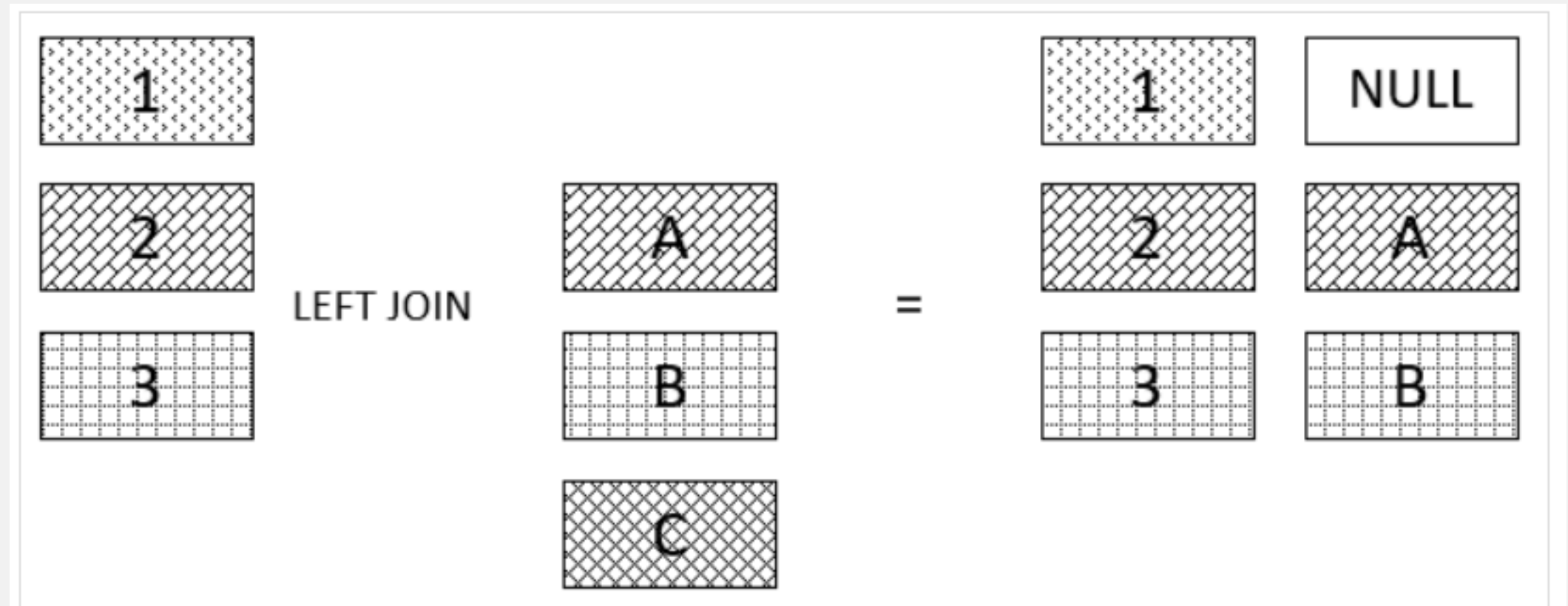
- **Thiết lập rất đơn giản:** thêm gói sqflite vào **pubspec.yaml** là có thể mở CSDL và thực thi SQL thô ngay lập tức.
- **Không cần build_runner:** nhanh hơn cho các thay đổi đơn giản vì chỉ cần viết mã và chạy.

4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.1. Truy vấn JOIN: Lấy Sản phẩm cùng với Tên Danh mục:

Yêu cầu: Hiển thị một danh sách sản phẩm, mỗi sản phẩm đi kèm với tên của danh mục tương ứng. Điều này yêu cầu một phép JOIN giữa bảng **products** và **categories**.

```
1 SELECT
2   select_list
3 FROM
4   T1
5 LEFT JOIN T2 ON
6   join_predicate;
```



4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.1. Truy vấn JOIN: Lấy Sản phẩm cùng với Tên Danh mục:

Sqflite: cách hiệu quả nhất để thực hiện phép JOIN là sử dụng **db.rawQuery** để thực thi một câu lệnh SQL thô.

```
1 Future<List<Map<String, dynamic>>> getProductsWithCategoryName() async
2   final db = await database;
3   final List<Map<String, dynamic>> maps = await db.rawQuery('''
4     SELECT
5       p.id,
6       p.name,
7       p.price,
8       c.name as category_name
9     FROM products p
10    LEFT JOIN categories c ON p.category_id = c.id
11  ''');
12   return maps;
13 }
```

Không an toàn kiểu:

Phân tích kết quả thủ công:

Kết quả trả về là

List<Map<String, dynamic>>,
phải tự phân tích Map này để tạo ra các đối tượng Dart.

4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.1. Truy vấn JOIN: Lấy Sản phẩm cùng với Tên Danh mục:

Drift: cung cấp hai cách để xử lý JOIN: thông qua **API Dart** an toàn kiểu hoặc định nghĩa trong tệp **.drift**.

Cách 1: Sử dụng API Dart:

Truy vấn trả về một danh sách:
Để dàng trích xuất các đối tượng bằng phương thức **row.readTable()**, loại bỏ hoàn toàn việc phân tích Map thủ công.

```
1 // Định nghĩa một lớp để chứa kết quả kết hợp
2 class ProductWithCategory {
3   final Product product;
4   final Category category;
5
6   ProductWithCategory({required this.product, required this.category});
7 }
8
9 // Trong lớp AppDatabase
10 Future<List<ProductWithCategory>> getProductsWithCategory() {
11   final query = select(products).join([
12     leftOuterJoin(categories, categories.id.equalsExp(products.categoryId))
13   ]);
14
15   return query.get().then((results) {
16     return results.map((row) {
17       return ProductWithCategory(
18         product: row.readTable(products),
19         category: row.readTable(categories),
20       );
21     }).toList();
22   });
23 }
```

4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.1. Truy vấn JOIN: Lấy Sản phẩm cùng với Tên Danh mục:

Drift: cung cấp hai cách để xử lý JOIN: thông qua **API Dart** an toàn kiểu hoặc định nghĩa trong tệp **.drift**.

Cách 2: Sử dụng tệp:

Tạo tệp queries.drift:

Drift tự động sinh ra một phương thức **getProductsWithCategoryDrift()** trả về:

**Future<List<ProductWithCategory
ViewData>> .**

```
import 'database/tables.dart';  
1  
2 //Định nghĩa một cấu trúc dữ liệu cho kết quả trả về  
3 CREATE VIEW ProductWithCategoryView AS SELECT  
4   p.**,  
5   c.name AS categoryName  
6 FROM products p  
7 LEFT JOIN categories c ON p.category_id = c.id;  
8  
9 //Định nghĩa một truy vấn có tên  
10 getProductsWithCategoryDrift: SELECT * FROM ProductWithCatego-  
11 ryView;
```


4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.2. Truy vấn Tổng hợp: Đếm xem mỗi User có bao nhiêu Todo.

Sqflite:

```
1 Future<List<Map<String, dynamic>>> getTodoCounts() async {
2   return await db.rawQuery(
3     ...
4     SELECT u.name, COUNT(t.id) as todoCount
5     FROM users u
6     LEFT JOIN todos t ON u.id = t.user_id
7     GROUP BY u.id
8     ...
9   );
10  // Kết quả: [{'name': 'Alice', 'todoCount': 5}, {'name': 'Bob', 'todoCount':
11  0}]
12  // Phải tự parse 'todoCount'
13 }
```

4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.2. Truy vấn Tổng hợp: Đếm xem mỗi User có bao nhiêu Todo.

Drift:

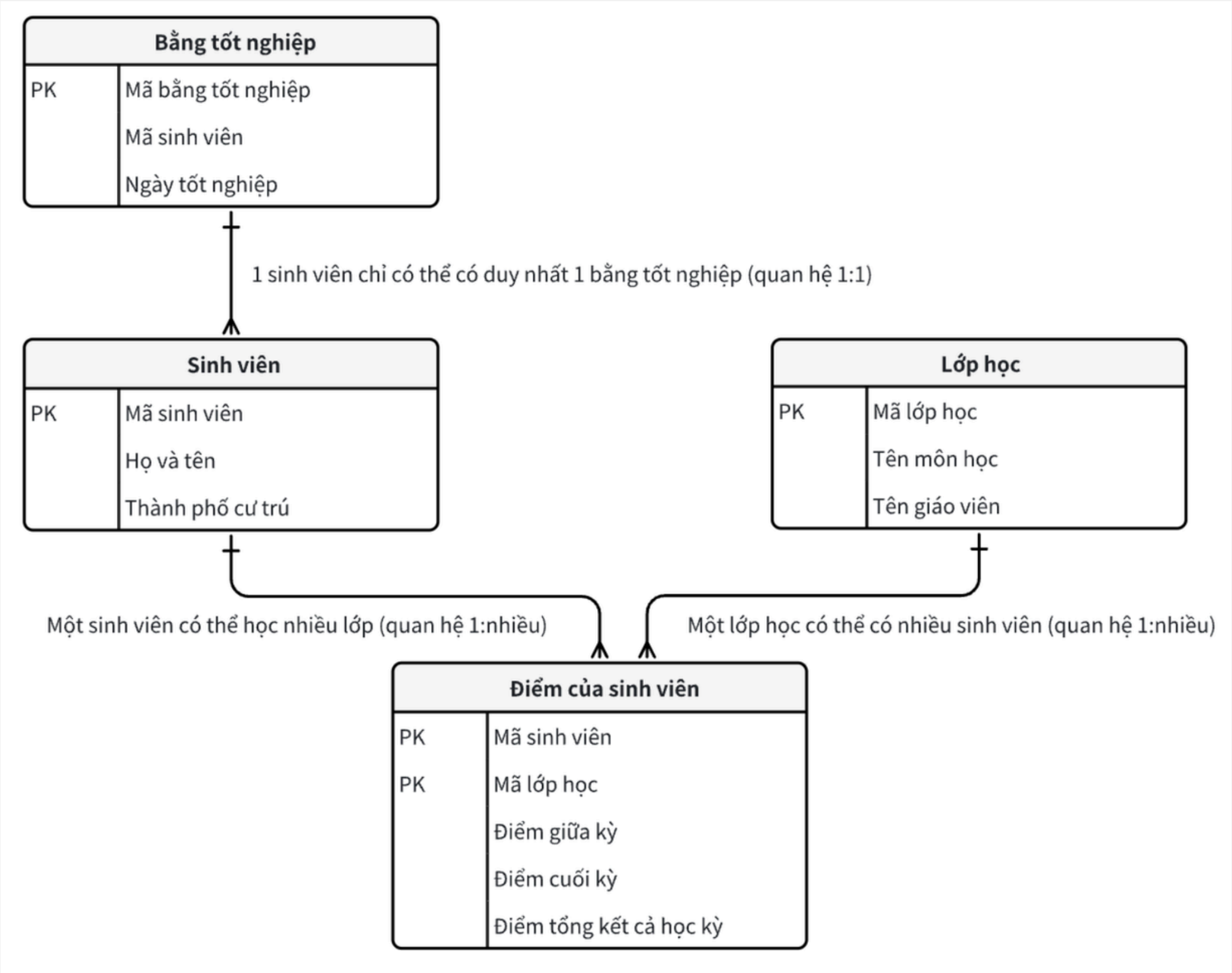
```
1 // Định nghĩa một lớp chứa kết quả
2 class User_TODO_Count {
3   final String userName;
4   final int todoCount;
5   User_TODO_Count({required this.userName, required this.todoCount});
6 }
7
8 Future<List<User_TODO_Count>> get_TODO_Counts() {
9   // 1. Định nghĩa biểu thức COUNT
10  // Dùng `filter` để nó hoạt động chính xác với LEFT JOIN (đếm 0)
11  final count = todos.id.count(filter: todos.user.equalsExp(users.id));
12
13  // 2. Viết truy vấn
14  final query = select(users)
15    // 3. Thêm cột "COUNT" đã tính toán vào
16    .addColumn([count])
17    // 4. Thực hiện LEFT JOIN
18    .join([
19      leftOuterJoin(todos, todos.user.equalsExp(users.id))
20    ])
21  }
```

```
21 // 5. GROUP BY
22 ..groupBy([users.id]);
23
24 // 6. Map kết quả
25 return query.get().then((rows) {
26   return rows.map((row) {
27     return User_TODO_Count(
28       userName: row.readTable(users).name,
29       // Đọc giá trị từ biểu thức `count`
30       todoCount: row.read(count) ?? 0,
31     );
32   }).toList();
33 });
34 }
```

4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.2. Mối quan hệ Many-to-Many:

Một sản phẩm có thể có nhiều thẻ, và một thẻ có thể được gán cho nhiều sản phẩm.



4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.2. Mối quan hệ Many-to-Many:

Sqflite: Quá trình này khá phức tạp và thủ công:

1. Tạo hai bảng mới: bằng các câu lệnh **CREATE TABLE SQL** thô.
2. Thêm một thẻ vào một sản phẩm: cần thực hiện một lệnh **INSERT** vào bảng **product_tags**.
3. Lấy tất cả các thẻ của một sản phẩm: cần thực hiện một truy vấn **JOIN** phức tạp qua **ba bảng (products, product_tags, tags)**.
4. Cập nhật các thẻ: cần nhiều thao tác **DELETE** và **INSERT** riêng lẻ.

4. Demo các truy vấn phức tạp, mối quan hệ của sqflite và Drift

4.2. Mối quan hệ Many-to-Many:

Drift: Dễ dàng hơn.

1. Định nghĩa bảng: Định nghĩa các bảng **Tags** và **ProductTags** bằng các lớp Dart, tương tự như **Products** và **Categories**.
2. Viết truy vấn:
 - Việc **INSERT/DELETE** được thực hiện qua API an toàn kiểu của Drift.
 - Để lấy một sản phẩm cùng với danh sách các thẻ của nó, viết một truy vấn JOIN phức tạp.

5. Kiểm tra hiệu năng và tối ưu hóa:

5.1. Kiểm tra hiệu năng:

Hiệu năng của một truy vấn cơ sở dữ liệu có thể được chia thành hai phần:

- **Thời gian thực hiện:** Thời gian mà động cơ cần để tìm và trả về dữ liệu.
- **Thời gian Truyền dữ liệu & Ánh xạ:** Thời gian cần để chuyển dữ liệu từ lớp native sang Dart và chuyển đổi nó.



5. Kiểm tra hiệu năng và tối ưu hóa:

5.1. Kiểm tra hiệu năng:

sqlite:

- **Thời gian thực hiện rất nhanh:** Hầu hết thời gian thực thi là thời gian mà chính động cơ SQLite xử lý truy vấn.
- **Thời gian Ánh xạ khá lâu:** sqlite trả về dữ liệu dưới dạng **List<Map<String, dynamic>>**. Việc này rất nhanh. Tuy nhiên, tốn thời gian viết mã Dart để lặp qua danh sách này và chuyển đổi từng Map thành đối tượng.
- **Thao tác hàng loạt:** Nếu bạn **INSERT** 10.000 hàng bằng cách gọi **db.insert()** 10.000 lần trong vòng lặp for, hiệu năng sẽ cực kỳ tệ. Mỗi lệnh insert tốn rất nhiều thời gian ghi đĩa (I/O).

5. Kiểm tra hiệu năng và tối ưu hóa:

5.1. Kiểm tra hiệu năng:

Drift:

- **Thời gian thực hiện gần như ngay lập tức:** Nếu dùng **Dart query builder**, Drift mất một vài micro giây (μs) để dịch mã Dart đó thành chuỗi SQL.
- **Thời gian Ánh xạ nhanh:** Drift tự động chuyển đổi **List<Map>** thành **List<User>**.
- **Thao tác hàng loạt:** Hiệu năng tương tự sqflite. Drift cũng cung cấp API **db.batch()** để gộp các thao tác ghi vào một giao dịch duy nhất.

5. Kiểm tra hiệu năng và tối ưu hóa:

5.2. Tối ưu hóa:

Tối ưu hóa SQLite Chung (Áp dụng cho cả hai):

- **Luôn sử dụng batch hoặc transaction khi thực hiện nhiều thao tác liên tiếp:**
Đây là **tối ưu hóa quan trọng nhất**. Nó giảm I/O đĩa từ N lần ghi xuống còn 1 lần ghi duy nhất.
- **Sử dụng Index:** hãy tạo một **INDEX** trên các cột đó để dễ dàng lọc và sắp xếp.
- **Tránh SELECT:** Chỉ truy vấn các cột khi thực sự cần. Việc này giảm lượng dữ liệu phải đọc từ đĩa và truyền qua cầu nối Native-to-Dart.
- **Cấu hình PRAGMA:** Sử dụng chế độ **journal_mode = WAL** để cải thiện hiệu suất bằng cách cho phép các thao tác đọc và ghi diễn ra đồng thời.

5. Kiểm tra hiệu năng và tối ưu hóa:

5.2. Tối ưu hóa:

Sqflite:

- Sử dụng **db.batch()**: là API quan trọng nhất để tối ưu hóa ghi hàng loạt.
- Sử dụng **API noResult**: sử dụng **rawInsert** hoặc **executeUpdate** thay vì **rawQuery** để tránh thời gian tạo Map không cần thiết.

Drift:

- **Chạy trên luồng riêng**: Mặc định, các truy vấn chạy trên luồng chính của Flutter. Hãy khởi tạo CSDL trên một luồng nền .
→ **Kết quả**: Mọi truy vấn sẽ tự động chạy trên luồng nền. Luồng chính của Flutter không bao giờ bị chặn. Hiệu năng của ứng dụng tăng lên 100% vì UI luôn mượt mà.

Tài liệu tham khảo:

[1] **Drift Docs (Isolates):** <https://drift.simonbinder.eu/docs/advanced-features/isolates/>

[2] **SQLite Home Page:** <https://sqlite.org/>

[3] **Stack Overflow:** <https://stackoverflow.com/questions/56464953/flutter-sqlite-one-to-many-relationship-setup>

[4] **Flutter Performance Optimization:**
<https://medium.com/@trrev2021/flutter-performance-optimization-best-practices-techniques-e614b84df7eb>

[5] **Làm việc với cơ sở dữ liệu SQLite-CRUD:** <https://hanam88.com/kho-tai-lieu/63/282/flutter-can-ban-lam-viec-voi-co-so-du-lieu-sqlite-crud.html>